

1.

(a) $L_1 = \{a^{n!} \mid n \in \mathbb{N}\}$.

Assume to the contrary that L is context-free. Let $p \geq 1$ be the pumping length given by the pumping lemma for CFLs.

Let n s.t. $n! \geq p$. Consider $s = a^{n!} \in L_1$. $\therefore |s| = n! \geq p$.

$s = uvwxy$ s.t. $|vwx| \leq p$, $|vx| \geq 1$, $uv^iwx^iy \in L_1 \forall i \geq 0$.

Let $k = |vx|$. $\therefore |vwx| \leq p$, $1 \leq k \leq p$.

Pump s with $i=2$.

$$s' = uv^2wx^2y \Rightarrow |s'| = |s| + |vx| = |s| + k = n! + k.$$

~~For $s' \in L_1$~~

For s' to belong to L_1 , its length must be $(n+m)!$ for some $m \in \mathbb{N}$, $m \geq 1$.

The smallest possible length for s' longer than $a^{n!}$ is $a^{(n+1)!}$.

$$\therefore n! \geq p \geq k \geq 1,$$

$$n! < n! + k \leq n! + p \leq n! + n! < n! + n \cdot n! = (n+1)!$$

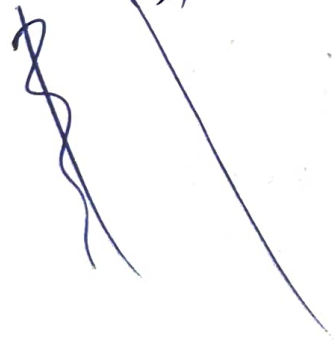
$$\therefore n! < n! + k < (n+1)!$$

$\therefore |s'|$ lies strictly between two consecutive factorials, $|s'| \neq m! \forall m \in \mathbb{N}$.

$\therefore s' \notin L_1$, contradicting the pumping lemma for CFLs.

$\therefore L_1$ is not context-free.

(b)



To decide L_1 , the TM computes factorials using two counters: N (current integer) and $M = N!$.

Algorithm:

Base Cases:

If input is ϵ , reject.

Mark 1st a . If no more a 's, accept ($0! = 1! = 1$).

Mark 2nd a . If no more a 's, accept ($2! = 2$).

Loop: Set $N=2$, $F=2$.

Loop N times:

Mark F un-marked a 's on tape.

If no more a 's are found mid-loop, reject.

~~Set $F =$~~

If all a 's are marked after the inner loop, accept.

(Else, set $N \leftarrow N+1$, $F \leftarrow (N+1)!$).

Repeat Loop.

$\therefore$ input length strictly decreases, $N!$ grows exponentially, TM halts on all inputs in finite steps.

(b) $L_2 = \{ ww^k w \mid w \in \{0,1\}^* \}$

Assume, to the contrary, that L_2 is context-free with pumping length p .

Choose $w = 0^p 1^p$. Then $w^k = 1^p 0^p$.

Then, $s = 0^p 1^{2p} 0^{2p} 1^p \in L_2$.

$\therefore |s| = 6p > p$, $s = uvwxy$ s.t. $|vwx| \leq p$, $|vx| \geq 1$.

$|vwx| \leq p$, $|vx| \geq 1$, $uv^iwx^iy \in L_2 \forall i \geq 0$.

$\therefore |vwx| \leq p$, it can span at most two adjacent block types in the blocks $0^p, 1^{2p}, 0^{2p}, 1^p$.

Case 1: $vw^i x$ falls in the first half of s .

Pumping $i=2$ alters n^2 of 0's, 1's in first w and/or w^R , but leaves second w untouched. s' cannot be decomposed to $uv^R u$.

Case 2: $vw^i x$ falls in the second half of s .

Pumping $i=2$ alters n^2 of 0's, 1's in w^R and/or second w , but leaves first w untouched. s' cannot be decomposed to $uv^R u$.

Case 3: $vw^i x$ straddles the boundary $0^{2p} 1^{2p} 0^{2p}$.

vx contains only 1's from ~~middle~~ ^{2nd} block and 0's from 3rd block. Pumping $i=0$ reduces $1^{2p} 0^{2p}$ segment w/o affecting 0^p or 1^p segments. ~~The~~ s' cannot be decomposed to $uv^R u$.

In all cases, the pumping lemma for CFL is violated and L_2 is hence not context-free.

To decide L_2 , the TM verifies $|s| \equiv 0 \pmod{3}$, identifies $|w| = \frac{|s|}{3}$,

Algorithm:

Scan tape left-to-right, marking off symbols in groups of 3 with temporary markers.

If $|s| \not\equiv 0 \pmod{3}$, reject.

Restore marked symbols to original values.

Place boundary markers for indices $k+1$, $2k+1$, shifting data forward. This delineates w , w^R , w .

for $i=0$ to $i=k-1$

read i^{th} symbol of B_1 at $x = s[1+i]$

compare x with $y = s[2k-i]$. if $x \neq y$, reject.

compare x with $z = s[2k+1+i]$. if $x \neq z$, reject.

mark $s[1+i]$, $s[2k-i]$, $s[2k+1+i]$ as processed.

if all iterations match, accept.

Since all ~~tape~~ loops iterate over finite tape lengths bounded by $|s|$,
the TM halts and decides L_2 .

2. $L = \{ w \# w' \mid w, w' \in \{U, D, L, R\}^*, w' = w, w \text{ returns to the origin from the origin} \}$.

~~$\therefore L = \{ w \# w \mid w \in \{U, D, L, R\}^*, w \text{ goes from the origin to the origin} \}$~~

Pumping Lemma for CFLs:

If L is context-free, there is a pumping length p where if $s \in L$ and $|s| > p$, then s may be divided into five strings

$s = uvxyz$ satisfying

$$\forall i > 0, uv^i xy^i z \in L$$

$$|vy| > 0$$

$$|vxy| \leq p$$

Let p be the pumping length for L .

Let $w = U^p R^p D^p L^p$, so $s = U^p R^p D^p L^p \# U^p R^p D^p L^p$.

Case 1. vxy is wholly in w .

Let $|vy| = k$. Now, ~~$s = uv^i xy^i z$ has length $|s| = 1$~~

Now, the left string w after pumping s with $i=2$ is longer than w' by k , so $w \neq w'$, so pumping leads to a string not in

L . Hence $vxy \notin w$.

Case 2. vxy is wholly in w' .

Let $|vy| = k$.

Now, the right string w' after pumping s with $i=2$ is longer than

w by k , so $w \neq w'$. So, pumping leads to a string not in L . Hence, $vxy \notin w'$.

Case 3: vxy has no part of w' , but may contain portions of w and the delimiter.

If $\#$ is in vxy , then $\#$ is in x , otherwise, pumping with $i=0$ yields a string without a delimiter, ~~hence~~.

Now, since vxy has no part of w' , and y must be after the delimiter, then $|y|=0$. But if $|v| \neq |y|$, then. Then, if $|v|=k$, pumping

with $i=2$, ~~but~~ w will have length k more than w' , so $w \neq w'$.

Hence, pumping leads to a contradiction.

Case 4: vxy has no part of w , but may contain portions of w' and the delimiter.

If $\#$ is in vxy , then $\#$ is in vxy . otherwise, pumping with $i=0$ yields a string without a delimiter.

Now, since vxy has no part of w and y must be before the delimiter, then $|y|=0$. Then, if $|y|=k$, pumping with $i=2$, w' will have length k more than w , so $w \neq w'$.

Hence, pumping leads to a contradiction.

Case 5: vxy ~~may~~ has portions of w , w' , and has the delimiter.

~~If $|v| \neq |y|$,~~

If the delimiter is in v or y , pumping with $i=0$ produces a string with no delimiter, hence, $\#$ is in x .

Hence, v is wholly within w , and y is wholly within w' .

If $|v| \neq |y|$, then pumping with $i=2$ would give w and w' with different lengths, which is a contradiction. $\therefore |v| = |y|$.

Now, since $|vxy| > 0$, $|v| > 0$, $|y| > 0$.

Since $|vxy| \leq p$ and $|y| > 0$, $|vx| < p$. ~~Now, no matter where~~

$\therefore |x| \geq 1$ (contains delimiter), $|v| < p$. $\therefore v$ cannot

extend more than p positions left of $\#$. Therefore, ~~pumping~~

v is composed solely of L 's. Pumping with $i=2$ gives

$w = U^p R^p D^p L^{p+|v|}$, which does not land at the origin.

Hence, a contradiction.

Hence, in all cases, a contradiction is reached. So, L is not a CFL.

Q3.

a) Let R be a decider for $L_1 = \{M : L(M) \text{ contains no palindromes}\}$.

Let S be a Turing machine ~~accept~~ operating as follows, ~~accepting~~ on input x .

1. Construct M_2 , a Turing machine operating as follows.

1. Simulate a Turing machine M on an input w and accept if M halts on w , else, reject.

2. Run R on $\langle M_2 \rangle$. If M halts on w , then M_2 accepts regardless of input ~~to S~~ , so $L(M_2) = \Sigma^*$, which contains strings containing palindromes, so R rejects. If M does not halt on w , then M_2 rejects regardless on input, so $L(M_2) = \emptyset$, which contains no strings containing palindromes, so R accepts. $\therefore R$ accepts iff M does not halt on w .

3. If R accepts $\langle M_2 \rangle$, reject, else, accept.

Now,

Now, S accepts iff R rejects iff M halts. So S decides HALT_{TM} .

$\therefore$ if such a decider R existed, S could decide HALT_{TM} . But

HALT_{TM} is undecidable, hence $P L_1$ is undecidable.

b) Let R be a decider for $L_2 = \{M : L(M) \text{ is regular}\}$.

Let S be a Turing machine operating as follows, ~~accepting~~ on input x .

1. Construct M_2 , a Turing machine operating as follows.

1. If the input x has the form $0^n 1^n$, accept.

2. Else, accept if the Turing machine M halts on input w , else reject.

2. Run R on $\langle M_2 \rangle$. If M halts on w , then M_2 accepts, so $L(M_2)$ is then Σ^* , a regular language. If M does not halt on w , then M_2 accepts iff the input is $0^n 1^n$, a non-regular language.

Hence, R accepts M_2 iff M halts on w .

3. If R accepts, accept. If R rejects, reject.

Now, S accepts if R accepts M_2 iff M halts on w .

Hence, if such a decider R existed, we could construct a decider for HALT_{TM} . But HALT_{TM} is undecidable, hence no such R exists.

b) The empty language ϕ either belongs to P or it does not. Since P is non-trivial, there is an RE language in P and one not in P .

Let R_P be a decider for L_P .

If $\phi \in P$, choose A to be a TM language $A_n \notin P$, and if $\phi \notin P$, choose $A \in P$ s.t. $L(A) \in P$.

Construct M_2 , a TM, as operating as follows with input x .

1. Simulate ~~M~~ a TM M on input w .

2. If $M(w)$ halts, run $A(x)$, accept if $A(x) \in L(A)$, else reject.

Run $R_P(\langle M_2 \rangle)$. if ~~M halts loops~~ $M(w)$ loops forever,

$L(M_2) = \phi$. Else, $L(M_2) = L(A)$. Now, if ~~R_P~~ the result

of R_P depends only on whether $M(w)$ halts, so R_P can

be used to decide HALT_{TM} . But HALT_{TM} is undecidable,

so R_P L_P is undecidable.

In essence, select two languages, only one of which is in P , and the other is ϕ one of which is ϕ . Let the other be denoted $L(A)$, w/ TM A .

The language of a machine is ϕ if it always loops -

so if ~~$M(w)$ loops~~, so by tying whether the TM A runs to whether $M(w)$ halts, the language is tied being in P is tied to HALT_{TM} .

c) ~~IF~~ ~~M~~

For L_1 :

Let M_1, M_2 be ~~machines~~ TMs.

If $L(M_1) = L(M_2)$, both or neither contains palindromes.

$\emptyset$ contains no palindromes, and Σ^* contains palindromes.

$\therefore$ the property "the language contains no palindromes" is non-trivial and semantic, so it is undecidable.

For L_2 :

Let M_1, M_2 be TMs.

If $L(M_1) = L(M_2)$, both or neither are regular.

$\{0^n 1^n \mid n \in \mathbb{N}\}$ is nonregular, Σ^* is regular.

$\therefore$ the property "the language is regular" is non-trivial and semantic, so it is undecidable.

4. Let D be a ~~decider~~ for Turing machine accepting input $\langle M, w \rangle$ operating as follows:

1. ~~Simulate M on w step~~ - Initialise a step counter $c = 0$.

2. Simulate M on w step-by-step.

3. For every step of M :

1. Increment c by 1.

2. If M enters an accept or reject state, accept $\langle M, w \rangle$.

3. If $c = 100$ and M has not halted, reject $\langle M, w \rangle$.

For any ~~valid~~ ~~input~~ ~~in~~ $\langle M, w \rangle$, since there is a bijection between finite strings, naturals, and TMs, no input need strictly be treated as malformed. Because each step executes in finite time and there are a finite number of steps, D always halts, hence, decides L .

5. A non-deterministic Turing machine can simulate a 2-stack PDA P where $P = (Q_P, \Sigma, \Gamma, \delta_P, q_{0P}, F_P)$

Let, initially, the two-way infinite tape be structured as follows.

The ~~input~~ tape contains the input string.

On either side of the input string, let there stack empty symbol $\$$ be the ~~be inserted~~.

One stack grows left, with its $\$$ being the one left of input string.

One stack grows right, with its $\$$ being the one right of input string.

The tape head is initially at the first input symbol.

Let a transition in the 2-stack PDA be given by

$Q_0(a, b | c, d$

$q_0(a, b \rightarrow c, d \rightarrow e) \Rightarrow \{q_i, \dots, q_j\}$

where a symbol a is read from input when in state q_0 ,

b is popped and c is pushed in first stack, d is popped

and e is pushed in second stack, and PDA transitions

non-deterministically to one of $q_i, \dots, q_j$.

This transition is simulated as

the NTM in q_0 ~~rea~~ moves the ~~no~~ tape head to read a . It then tracks left until it sees a blank (indicating it has gone past stack 1 top, moves right, and overwrites b with c (if $c = \epsilon$, write $\sqcup$), ~~then moves right until it sees a blank or if~~

$b = \epsilon$, writes c without moving right, a similar procedure

with direction flipped for stack 2 on the right, then moves tape head to next input symbol, and moves

to one of $q_i, \dots, q_j$ non-deterministically.

If $\mathcal{P}$ the NTM enters a accepting state $q \in F$, it transitions to q_{accept} and halts. Else, if entering a ~~reject~~ halting non-accepting state, it transitions to q_{reject} .

An NTM can be simulated by a DTM, so a DTM with 2-way infinite tape decides $L(P)$.

Let T be a deterministic TM $(Q_T, \Sigma, \Gamma, \delta_T, q_0, q_{\text{accept}}, q_{\text{reject}})$ w/ a single two-way infinite tape.

We construct a 2-PDA P with stacks S_1 and S_2 simulating T .

S_1 (left stack) stores all written symbols to the left of T 's tape head, with the top being the cell immediately left of tape head.

S_2 (right stack) stores all written symbols to the ~~left of~~ under and to the right of T 's tape head, top being cell under tape head.

Given input $w \neq \epsilon = w_1 w_2 \dots w_n \in \Sigma^+$,

P pushes $\$$ (bottom-stack marker) in S_1, S_2 .

P reads w from input tape and pushes $w_n, w_{n-1}, \dots, w_1$ in that order to S_2 , so w_1 is top of S_2 . If $w = \epsilon$, P pushes $\perp$ to S_2 .

P enters state q_0 .

For each step of T in $q \in Q_T$,

P inspects top of S_2 (if $\$$, treat as $\perp$)

$\delta_T(q, x) = (q', y, D)$ is executed as:

P pops x from S_2 , pushes y to S_2 .

If $D = R$,

P pops y from S_2 , pushes y to S_1 .

If top of S_2 is $\$$, push $\perp$.

If $D = L$

P pops z from S_1 . If top of S_1 is $\$$, ~~push $\perp$~~ let $z = \perp$

P pushes z to S_2 .

If $q' = q_{\text{accept}}$, P enters an accepting state.

If $q' = q_{\text{reject}}$, P halts w/o accepting.

$\therefore$ ~~P decides~~ P decides $L(T)$.

$\therefore L(P) = L(T)$.

Any 2-PDA is a k -PDA having $k-2$ stacks unused.

A k -PDA can be simulated by a TM dividing tape into k regions.

A TM is simulatable by a 2-PDA.

$\therefore 2\text{-PDA} \subseteq k\text{-PDA} \subseteq \text{TM} \subseteq 2\text{-PDA}$.

$\therefore 2\text{-PDA} = k\text{-PDA} = \text{TM}$.

6.

a). If $C_T(x) = \infty$, then $C_U(x) \leq \infty$ is trivially true.

If $C_T(x)$ is finite, there exists a shortest input p s.t. $T(p) = x$.

To have U output x , ~~we~~ since U is a universal Turing machine, we can have U simulate T on x . This requires the input $\langle T, p \rangle$, where $\langle T \rangle$ is the encoding of the Turing machine T , and p is the shortest input p s.t. $T(p) = x$.

This input has length $|\langle T \rangle| + |p|$.

But $C_T(x) = |p|$, so $C_U(x) = |\langle T \rangle| + C_T(x)$

$|\langle T \rangle|$ is a constant depending only on T . Hence,

$$b_T := |\langle T \rangle|$$

$\therefore C_U(x) \leq b_T + C_T(x)$ (since ~~any~~ we only need an upper bound, there might yet be smaller input)

b) Let T be a Turing machine that does no processing on the input, but halts immediately s.t. its output is its input. Then, ~~$T(p) = p$~~ $T(x) = x$. $\therefore C_T(x) \leq |x| = n$

$$\therefore C_U(x) \leq b_T + C_T(x) \leq b_T + n$$

b_T does not depend on x or n , since $b_T = |\langle T \rangle|$.

$$\therefore b_T \in O(1)$$

$$\therefore C_U(x) \leq n + O(1)$$

c) From part (a),

since U is a universal Turing machine,

$$C_U(x) \leq b_U + C_V(x), \therefore \{C_U(x) - C_V(x)\} \leq b_U$$

since V is a universal Turing machine,

$$C_V(x) \leq b_V + C_U(x) \therefore \{C_U(x) - C_V(x)\} \geq -b_V$$

$$\therefore -b_u \leq C_u(x) - C_v(x) \leq b_v$$

$$\text{Let } c := \max(b_u, b_v).$$

$$|C_u(x) - C_v(x)| \leq c$$

(d) b_k is a binary string composed of a 1 followed by $f(k-1)$ zeros, for $k > 1$, and $b_1 = 10$.

This string is generatable with only the input k .

Algorithm:

Read k from the tape.

Compute $f(k)$ ~~iteratively~~ recursively as $f(k-1) 2^{f(k-2)}$, with the base case $k=1$ returning 10.

Return 1 followed by $f(k-1)$ zeros on the tape.

The program size is independent of k .

If a universal Turing machine is to execute this algorithm to produce b_k , it must receive as input the Turing machine T that executes the algorithm and the integer k .

The size of this input is as follows:

The description of the Turing machine executing the algorithm does not grow with k . It is constant, i.e., $O(1)$.

The input integer k can be expressed as a binary string in base-2, with length $\leq \log_2 k + 1$.

$$\therefore C(b_k) \leq O(1) + \log k + 1$$

$$\leq \log k + O(1)$$

(e) There are 2^p inputs (programs) of size p , and each program when simulated on a universal Turing machine must produce exactly one output string each.

Consider the set of strings that are not c -incompressible.

$$C(x) \leq |x| - c + 1$$

The number of programs of length at most $|x| - c + 1$ is

$$2^0 + 2^1 + 2^2 + \dots + 2^{n-c+1} = 2^{n-c} - 1$$

$\therefore$, The number of strings generatable by programs of length at most $n-c+1$ is $\leq 2^{n-c} - 1$ (duplicates may exist)

The number of strings that are c -incompressible are then

$$\begin{aligned} 2^n &\leq 2^n - 2^{n-c} + 1 \\ \text{(Total number of strings)} & \end{aligned} \quad \square$$

7. (a) Let p_x be the shortest program that outputs x when run on U .

$\forall n \in \mathbb{Z}, n \geq 1$, we can construct 2^n distinct valid programs of length $|p_x| + n$ that output x by appending n useless bits that are ignored while executing p_x .

$$\text{Then } P(x) = \sum_{p: U(p)=x} 2^{-|p|}$$

$$= 2^{-|p_x|} + 2 \times 2^{-(|p_x|+1)} + 2^2 \times 2^{-(|p_x|+2)} + \dots$$

$$= \sum_{n=1}^{\infty} \sum_{\substack{p: U(p)=x \\ |p|=|p_x|+n}} 2^{-|p|}$$

$$= \cancel{2^{-|p_x|}} + \dots$$

$$\geq \sum_{n=1}^{\infty} 2^n 2^{-(|p_x|+n)} = \sum_{n=1}^{\infty} 2^{-|p_x|} = \infty$$

$\therefore P(x) = \infty \forall x$, and hence such a probability measure cannot exist for the standard Turing machine model.

b) Let p, q be two inputs on which T halts successfully.
 Further, wlog, $p = qs$ for some non-empty $s \in \{0,1\}^*$, $|s| > 0$.
 Because $T(p)$ is a successful computation, T halts with the tape head of the input tape on the last bit of the input p , i.e., at position $|p|$, it enters a halting state.
 When executing q and T is deterministic, the execution at up to ~~the~~ position $|p|$ is identical with the execution of p .
 At position $|p|$, T enters a halting state. But this is not at position $|q|$, which is the last bit of q . Hence the computation of q fails. This is a contradiction, ~~since~~.
 Hence, $p \neq qs \forall s \in \{0,1\}^*, |s| > 0$.

$\therefore$, the set of inputs p for which T halts is prefix-free.
 Now, $\forall x$, the set ~~$\{p \mid U(p) = x\}$~~ $\{p \in \{0,1\}^* \mid U(p) = x\}$ is a subset of the set of inputs for which T halts (either accepts or rejects), which is prefix-free, and hence,
 $\forall x \{p \in \{0,1\}^* \mid U(p) = x\}$ is prefix-free.

c) Let $B = b_1, b_2, \dots$ be a sequence an infinite binary sequence, where

$$P(b_i = 0) = P(b_i = 1) = \frac{1}{2}$$

If E_x is the event that B is a prefix of x , then

$$\begin{aligned} P(E_x) &= P(b_1 = x_1) \times P(b_2 = x_2) \times \dots \times P(b_{|x|} = x_{|x|}) \\ &= \left(\frac{1}{2}\right)^{|x|} = 2^{-|x|} \end{aligned}$$

$$P\left(\bigcup_{x \in S} E_x\right) = \sum_{x \in S} P(E_x) = \sum_{x \in S} 2^{-|x|} \quad \text{iff}$$

$$\forall x, y \in S, E_x \cap E_y = \emptyset \text{ if } x \neq y.$$

i.e., we must prove that if x, y in S , neither can be true simultaneously.

If $\bar{E}_x \therefore S$ is prefix-free, x cannot be a prefix of y , and neither y of x .

If E_x , then x is a prefix of B .

If $E_x \cap E_y \neq \emptyset$, then x is a prefix of B and y is a prefix of B , so either x or y must be a prefix of the other.

$$\therefore E_x \cap E_y = \emptyset$$

$$\therefore P\left(\bigcup_{x \in S} E_x\right) = \sum_{x \in S} P(E_x) = \sum_{x \in S} 2^{-|x|}$$

By probability axioms, the ~~max~~ probability of a disjoint union of events in a sample space never exceeds 1.

$$\therefore P\left(\bigcup_{x \in S} E_x\right) \leq 1$$

$$\Rightarrow \sum_{x \in S} 2^{-|x|} \leq 1$$

d) Let $S_x = \{p \in \{0,1\}^* \mid p \cup (p) = x\}$.

By (b), S_x is prefix-free.

By Kraft's inequality, $\sum_{p \in S_x} 2^{-|p|} \leq 1$

$$\sum_{x \in \{0,1\}^*} P(x) = \sum_{x \in \{0,1\}^*} \sum_{p: U(p)=x} 2^{-|p|}$$

Since every program p produces at most one output x ,

$S_x \cap S_y = \emptyset$ (no program can deterministically produce x and y)
(when $x \neq y$)

$$\therefore \sum_{x \in \{0,1\}^*} P(x) = \sum_{p \in \bigcup_{x \in \{0,1\}^*} S_x} 2^{-|p|}$$

$\bigcup_{x \in \{0,1\}^*} S_x$ denotes the set of $\{p \in \{0,1\}^* \mid U(p) \text{ halts successfully}\}$.

We showed in part (b) that the set of inputs p for which a Turing machine halts is prefix-free.

$$\therefore \sum_{p \in \bigcup_{x \in \{0,1\}^*} S_x} 2^{-|p|} \leq 1, \text{ by Kraft's inequality.}$$

$\therefore$, $P(x)$ defined on the self-delimiting Turing machine is a valid probability measure.